Oracle SQL + PL/SQL

Combined Study Notes: Sales Aggregation & Slowly Changing Dimensions (SCD)

These notes focus on understanding the concepts, why they are used, and the important SQL/PLSQL statements. The SCD section is theory-focused and avoids lengthy procedures.

Part 1: Sales Aggregation Using Date Variables

1. What is sales aggregation?

Sales aggregation means summarizing sales records to calculate useful business information, such as total revenue, total quantity sold, average order value, and highest or lowest sale. Aggregate functions summarize records instead of examining each one individually.

- How much revenue did we generate in October?
- How many orders were placed between two dates?
- Which category generated the highest sales?
- What was the total sales amount on a particular day?

2. Aggregate functions

Function	Purpose	Example
SUM()	Calculates the total	SUM(amount)
COUNT()	Counts records	COUNT(*)
AVG()	Calculates the average	AVG(amount)
MIN()	Finds the smallest value	MIN(amount)
MAX()	Finds the largest value	MAX(amount)

Example: summarize the retail_sales table into one result row.

```
SELECT
SUM(amount) AS total_sales,
COUNT(*) AS total_orders,
AVG(amount) AS average_sales,
MIN(amount) AS minimum_sale,
MAX(amount) AS maximum_sale
FROM retail_sales;
```

3. Filtering records using date variables

Date variables let a procedure receive dates from its caller instead of hard-coding them. This query demonstrates a range from October 1 through October 10, 2026.

```
SELECT SUM(amount)
FROM retail_sales
WHERE sale_date >= TRUNC(
TO_DATE('01-10-2026', 'DD-MM-YYYY')
)
AND sale_date < TRUNC(
TO_DATE('10-10-2026', 'DD-MM-YYYY')
) + 1;
```

Important: TO_DATE() converts text to an Oracle DATE; TRUNC(date) removes the time portion; using >= start date and < end date + 1 includes the whole ending day even if the column stores times.

4. How stored procedures help with sales reports

A stored procedure lets you reuse reporting logic with different inputs, such as p_start_date and p_end_date. It filters records, calculates totals, and displays results.

```
SELECT SUM(amount),
SUM(quantity),
```

```
COUNT(*)
INTO v_total_sales,
v_total_quantity,
v_total_orders
FROM retail_sales
WHERE sale_date >= TRUNC(p_start_date)
AND sale_date < TRUNC(p_end_date) + 1;
```

Concept: SELECT ... INTO stores query results in PL/SQL variables.

5. Types of sales reports

Report	What it tells us
Date-range report	Sales totals between two dates
Sales summary	Total, average, minimum, maximum and count
Daily sales report	Sales for one particular day
Category sales report	Sales for a category during a date range

Additional concepts to remember

- **NVL(value, 0)** can replace a NULL result with zero.
- **DBMS_OUTPUT.PUT_LINE()** displays messages or calculated values.
- **TO_CHAR()** formats dates and numbers for display.
- **GROUP BY** summarizes sales separately for each category or date.
- **COMMIT** saves database changes; a read-only sales report normally does not need it.

Part 2: Slowly Changing Dimensions (SCD)

1. What is SCD?

A Slowly Changing Dimension is a data warehousing technique for managing changes to descriptive data over time. Examples include a customer's city, an employee's department, a product's category, or a customer's membership level.

The key question is: when a value changes, should the database keep the old value or replace it? Different SCD types handle that requirement differently.

2. SCD Type 0 — Retain original value

Concept: The original value never changes. If a customer's registration city is Bangalore, it remains Bangalore even if the customer later moves to Hyderabad.

- Preserves the original value.
- Does not track subsequent changes to that attribute.
- Useful when the initial value must remain permanent.

Remember: Type 0 = Preserve.

3. SCD Type 1 — Overwrite the value

Concept: Update the existing record with the new value. For example, Bangalore is replaced with Hyderabad.

- The old value is overwritten.
- No history of the previous value is retained in the table.
- Useful for correcting errors or when historical information is unnecessary.

Remember: Type 1 = Replace.

4. SCD Type 2 — Maintain full history

Concept: Every change creates a new version row instead of overwriting the previous version. This supports historical reporting: what was true at a particular point in time?

Customer	City	Start date	End date	Current?
Arjun	Bangalore	Jan 1	Mar 31	No
Arjun	Hyderabad	Apr 1	Jul 31	No
Arjun	Pune	Aug 1	NULL	Yes

- **Surrogate key:** uniquely identifies each version.
- **Start date:** when that version becomes valid.
- **End date:** when that version stops being valid.
- **Current flag:** identifies the latest version.

Remember: Type 2 = Add rows and preserve history.

5. SCD Type 3 — Maintain limited history

Concept: Store the current value and previous value in separate columns. Example: current city = Hyderabad; previous city = Bangalore. After another change, current city = Pune; previous city = Hyderabad.

- Keeps the current and immediately previous values.
- Does not retain the complete sequence of older values.
- Useful when only a limited amount of history is required.

Remember: Type 3 = Add columns.

6. SCD Type 4 — Separate current and history tables

Concept: Store the latest information in one table and old values in a separate history table.

Current table	History table
Arjun | Pune	Arjun | Bangalore
	Arjun | Hyderabad

- Keeps current information separate from historical records.
- Makes the current-state table simpler to query.
- History details depend on how the history table is designed.

Remember: Type 4 = Separate history table.

7. Compare the five SCD types

Type	Storage approach	History
0	Keep original value	Original only
1	Overwrite existing row	None
2	Create new version rows	Full history
3	Store current and previous columns	Limited history
4	Separate current and history tables	Stored separately

Memory trick: 0 = Preserve; 1 = Replace; 2 = Rows; 3 = Columns; 4 = Separate table.

Part 3: Connecting the Two Topics

Concept	Sales aggregation	SCD
Main purpose	Summarize business transactions	Manage changes to descriptive data
Typical SQL	SUM, COUNT, AVG, date filters	INSERT, UPDATE, keys, date columns
Time-related concept	Filter transactions by a date range	Track when a dimension value was valid
Main output	Sales totals and summaries	Current information or historical versions
Typical use	Business reporting	Data warehouse dimension management

Final revision checklist

- ■ I understand SUM, COUNT, AVG, MIN and MAX.
- ■ I understand TO_DATE, TRUNC and inclusive date-range filtering.
- ■ I understand how SELECT ... INTO stores query results in variables.
- ■ I understand how stored procedures accept input parameters.
- ■ I can explain the difference between all five SCD types.
- ■ I understand why SCD Type 2 needs version rows and effective dates.

Final takeaway: Sales aggregation answers “What happened to our sales during this period?” SCD answers “How did our customer or product information change over time?” Understanding both is a strong foundation for SQL-based reporting and data warehousing.